ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



PHÁT TRIỂN ỨNG DỤNG INTERNET OF THINGS (CO3038)

BÁO CÁO BÀI THỰC HÀNH 2

LẬP TRÌNH FREERTOS TRÊN ESP32-S3

Lớp: L01 - Học Kỳ HK261

Ngày nộp: 27/10/2026

GVHD: Thầy Phan Văn Sỹ

Sinh Viên	MSSV
Phạm Công Võ	2313946

TP.Hồ Chí Minh, Tháng 10/2026

THÔNG TIN THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

BÀI THỰC HÀNH 2

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Phạm Công Võ	2313946	• Hiện thực toàn bộ các bài tập từ Bài 1 đến Bài 5 • Thực nghiệm trên Kit YoloUNO (ESP32-S3) / CoreIOT • Soạn thảo tài liệu báo cáo	100%	

Mục lục

1 Tổng Quan và Mục Tiêu Bài Thực Hành 2 **3**
1.1 Giới thiệu . . . 3
1.2 Mục tiêu bài thực hành . . . 3
1.3 Bảng ánh xạ phần cứng trên bo mạch YoloUNO (ESP32-S3) . . . 3

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành **4**
2.1 Bài 1: Nhấp nháy luân phiên đơn giản (LED SÁNG 1s / TẮT 2s) . . . 4
2.1.1 Biểu đồ thời gian hoạt động (Timing Diagram) . . . 4
2.1.2 Mã nguồn hoàn chỉnh (`exercise_1.cpp`) . . . 4
2.1.3 Phân tích kỹ thuật & Cơ chế FreeRTOS . . . 5
2.2 Bài 2: Phối hợp LED và NeoPixel (Đồng bộ qua biến dùng chung) . . . 6
2.2.1 Mô hình đồng bộ hóa dữ liệu liên tác vụ . . . 6
2.2.2 Mã nguồn hoàn chỉnh (`exercise_2.cpp`) . . . 6
2.2.3 Phân tích kỹ thuật & Cơ chế FreeRTOS . . . 7
2.3 Bài 3: Các mẫu nhấp nháy độc lập (Independent Blink Patterns) . . . 9
2.3.1 Biểu đồ thời gian bất đồng bộ (Asynchronous Timing) . . . 9
2.3.2 Mã nguồn hoàn chỉnh (`exercise_3.cpp`) . . . 9
2.3.3 Phân tích kỹ thuật & Cơ chế FreeRTOS . . . 10
2.4 Bài 4: Chuỗi màu NeoPixel nâng cao (Cầu vồng 5 màu, mỗi màu 3s) . . . 11
2.4.1 Bảng ánh xạ mã màu RGB cho chuỗi 5 màu cầu vồng . . . 11
2.4.2 Mã nguồn hoàn chỉnh (`exercise_4.cpp`) . . . 11
2.4.3 Phân tích kỹ thuật & Cơ chế FreeRTOS . . . 13
2.5 Bài 5: Mô phỏng hệ thống đèn giao thông với nhịp tim Heartbeat . . . 14
2.5.1 Mô hình máy trạng thái hữu hạn (FSM) Đèn Giao Thông . . . 14
2.5.2 Mã nguồn hoàn chỉnh (`exercise_5.cpp`) . . . 14
2.5.3 Phân tích kỹ thuật & Ý nghĩa hệ thống . . . 16

3 Kết Luận và Bài Học Kinh Nghiệm (Lab 2) **17**

Tài liệu tham khảo **18**

1 Tổng Quan và Mục Tiêu Bài Thực Hành 2

1.1 Giới thiệu

Bài thực hành 2 tập trung vào việc nghiên cứu và ứng dụng hệ điều hành thời gian thực **FreeRTOS** trên vi điều khiển 32-bit kiến trúc lõi kép **ESP32-S3** (bo mạch YoloUNO) trong môi trường phát triển PlatformIO / VS Code. Sinh viên được làm quen với các khái niệm nền tảng trong lập trình nhúng chuyên nghiệp:

- **Đa nhiệm chiếm ưu thế (Preemptive Multitasking):** Khả năng phân chia thời gian CPU cho nhiều luồng thực thi độc lập dựa trên mức ưu tiên (Priority).
- **Cơ chế định thời phi phong tỏa (`vTaskDelay`):** Đưa tác vụ vào trạng thái *Blocked* trong khoảng thời gian xác định, giải phóng tài nguyên CPU cho các tác vụ khác hoặc *Idle Task* nhằm tiết kiệm năng lượng.
- **Đồng bộ hóa dữ liệu liên tác vụ (Inter-Task Communication):** Sử dụng từ khóa `volatile` đối với các biến toàn cục dùng chung để đảm bảo tính toàn vẹn dữ liệu, ngăn ngừa trình biên dịch tối ưu hóa sai lệch qua thanh ghi CPU.
- **Thiết kế máy trạng thái hữu hạn (FSM):** Ứng dụng điều khiển hệ thống đèn giao thông nhiều pha kết hợp chỉ thị nhịp tim hệ thống (*Heartbeat*).

1.2 Mục tiêu bài thực hành

1. Khởi tạo và cấu hình tham số cho các tác vụ FreeRTOS thông qua hàm `xTaskCreate()` (gồm hàm thực thi, tên tác vụ, dung lượng Stack, tham số truyền, mức ưu tiên và task handle).
2. Điều khiển ngoại vi xuất tín hiệu số: LED đơn tích hợp trên bo mạch (GPIO 48) và đèn LED thông minh RGB NeoPixel WS2812B (GPIO 45).
3. Phối hợp và đồng bộ hóa trạng thái hoạt động giữa hai tác vụ chạy song song thông qua biến cờ dùng chung.
4. Xây dựng các tác vụ bất đồng bộ với chu kỳ hoạt động độc lập và máy trạng thái đèn giao thông tuần tự.

1.3 Bảng ánh xạ phần cứng trên bo mạch YoloUNO (ESP32-S3)

Tên ngoại vi	**Chân GPIO**	**Giao thức / Điện áp**	**Ý nghĩa điều khiển**
LED đơn tích hợp (On-board)	GPIO 48	Digital Output (3.3V)	Mức HIGH (1): SÁNG; Mức LOW (0): TẮT
LED RGB NeoPixel WS2812B	GPIO 45	One-wire NRZ (800 kHz)	Điều khiển màu sắc độc lập qua giao thức 24-bit GRB
Cổng nạp & USB Serial CDC	Native USB	UART / COM (115200)	Ghi nhận log thực nghiệm và nạp firmware

Bảng 1: Bảng ánh xạ chân ngoại vi sử dụng trong Lab 2

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành

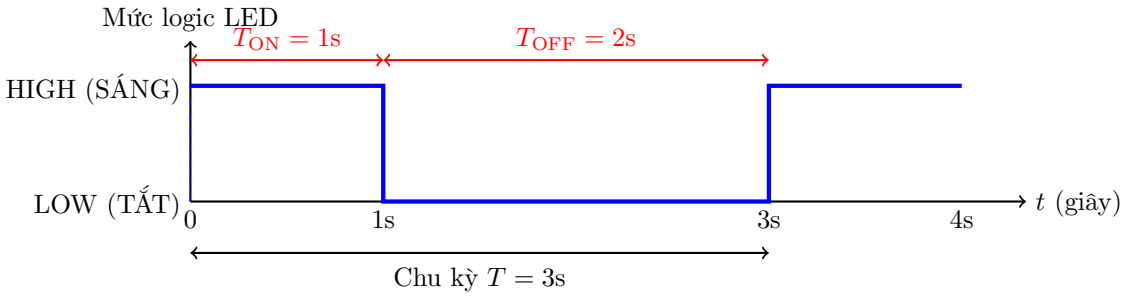
2.1 Bài 1: Nhấp nháy luân phiên đơn giản (LED SÁNG 1s / TẮT 2s)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Hiện thực tác vụ `task_led_blinky` sao cho LED trên board SÁNG trong 1 giây và TẮT trong 2 giây.**
- **Chỉ chạy duy nhất tác vụ này và quan sát LED nhấp nháy trên phần cứng.**

2.1.1 Biểu đồ thời gian hoạt động (Timing Diagram)

Chu kỳ hoạt động toàn phần là $T = T_{\text{ON}} + T_{\text{OFF}} = 1\text{s} + 2\text{s} = 3\text{s}$, với hệ số công tác (Duty Cycle) $D = \frac{1}{3} \approx 33.3\%$.



2.1.2 Mã nguồn hoàn chỉnh (`exercise_1.cpp`)

```cpp
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)

// Tac vu 1: Nhap nhay LED don (SANG 1 giay, TAT 2 giay)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);
  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(1000); // Sang 1 giay

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(2000); // Tat 2 giay
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(
      task_led_blinky,    // Ham tac vu
      "Task_LED_Blinky", // Ten tac vu
      2048,              // Stack depth (bytes)
      NULL,              // Tham so truyen vao
```

```
      1,                  // Do uu tien (Priority)
      NULL                // Task handle
  );
}

void loop() {
  // FreeRTOS quan ly tac vu, loop() de trong
}
```

2.1.3 Phân tích kỹ thuật & Cơ chế FreeRTOS

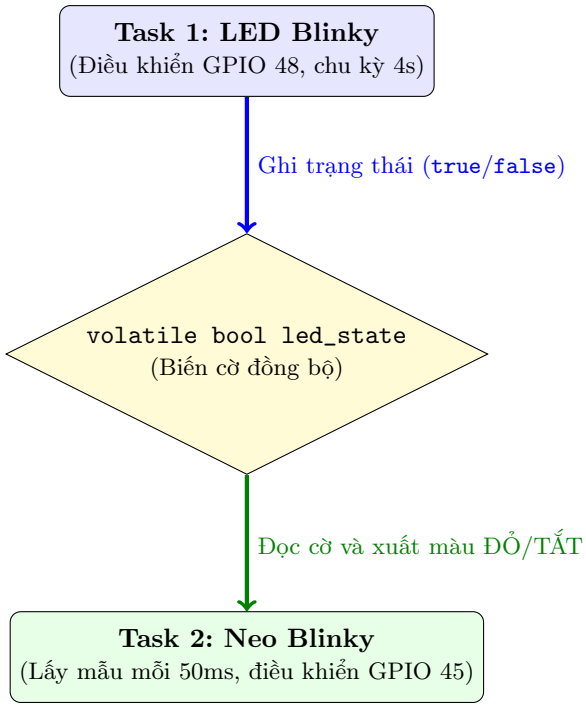
- **Cấu hình tác vụ:** Hàm `xTaskCreate()` cấp phát một khối bộ nhớ ngăn xếp (Stack) dung lượng 2048 bytes trong vùng nhớ Heap của ESP32-S3 để lưu trữ ngữ cảnh tác vụ (Task Context: thanh ghi CPU, biến cục bộ). Mức ưu tiên được đặt là 1.
- **Cơ chế định thời `vTaskDelay()`:** Trên ESP32 Arduino Core, tần số ngắt nhịp của hệ điều hành (`configTICK_RATE_HZ`) được thiết lập mặc định là 1000 Hz, tương đương mỗi tick hệ thống là 1 ms. Do đó, lời gọi `vTaskDelay(1000)` làm trễ đúng 1000 ticks (1 giây).
- **Giải phóng CPU:** Khác với hàm `delay()` truyền thống làm nghẽn luồng xử lý bằng vòng lặp bận (Busy-waiting), `vTaskDelay()` lập tức đưa tác vụ vào trạng thái *Blocked*. Bộ lập lịch (Scheduler) lập tức chuyển quyền điều khiển CPU cho các luồng nền (*IDLE task*) để thực hiện các công việc dọn dẹp bộ nhớ và tiết kiệm điện năng.

2.2 Bài 2: Phối hợp LED và NeoPixel (Đồng bộ qua biến dùng chung)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Hiện thực `task_led_blinky` với mẫu SÁNG 2 s, TẮT 2 s.**
- **Hiện thực `task_neo_blinky` để bật NeoPixel màu ĐỎ khi LED SÁNG, và TẮT (ĐEN) khi LED TẮT.**
- **Sử dụng một biến dùng chung để phối hợp và đồng bộ hóa giữa hai tác vụ.**

2.2.1 Mô hình đồng bộ hóa dữ liệu liên tác vụ



2.2.2 Mã nguồn hoàn chỉnh (`exercise_2.cpp`)

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)
#define NEO_PIN 45          // NeoPixel data pin (GPIO 45)
#define LED_COUNT 1         // 1 LED NeoPixel

// Bien chia se trang thai dong bo giua 2 tac vu
volatile bool led_state = false;

// Tac vu 1: Dieu khien LED don (SANG 2s, TAT 2s) va cap nhat bien led_state
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);
  while (1) {
    digitalWrite(LED_PIN, HIGH);
```

```
    led_state = true; // Bao trang thai LED dang SANG
    vTaskDelay(2000);

    digitalWrite(LED_PIN, LOW);
    led_state = false; // Bao trang thai LED dang TAT
    vTaskDelay(2000);
  }
}

// Tac vu 2: Dieu khien NeoPixel theo bien led_state (DO khi LED SANG, TAT khi LED TAT)
void task_neo_blinky(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    if (led_state) {
      strip.setPixelColor(0, strip.Color(255, 0, 0)); // Bat DO khi LED SANG
    } else {
      strip.setPixelColor(0, strip.Color(0, 0, 0));   // TAT khi LED TAT
    }
    strip.show();
    vTaskDelay(50); // Kiem tra lai sau 50ms
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(
      task_led_blinky,   // Ham tac vu
      "Task_LED_Blinky", // Ten tac vu
      2048,              // Stack depth (bytes)
      NULL,              // Tham so truyen vao
      1,                 // Do uu tien (Priority)
      NULL               // Task handle
  );

  xTaskCreate(
      task_neo_blinky,   // Ham tac vu
      "Task_NEO_Blinky", // Ten tac vu
      2048,              // Stack depth (bytes)
      NULL,              // Tham so truyen vao
      1,                 // Do uu tien (Priority)
      NULL               // Task handle
  );
}

void loop() {}
```

2.2.3 Phân tích kỹ thuật & Cơ chế FreeRTOS

- **Tầm quan trọng của từ khóa `volatile`:** Trong hệ thống đa nhiệm đa lõi, nếu không khai báo `volatile`, trình biên dịch (GCC) sẽ tối ưu hóa vòng lặp của `task_neo_blinky` bằng cách đọc biến `led_state` vào một thanh ghi CPU nội bộ và tái sử dụng giá trị đó liên tục. Từ khóa

`volatile` bắt buộc CPU phải đọc trực tiếp từ bộ nhớ RAM mỗi khi truy cập, đảm bảo thay đổi từ `task_led_blinky` được phản ánh ngay lập tức sang `task_neo_blinky`.

- **Tần số lấy mẫu trong tác vụ NeoPixel:** `task_neo_blinky` thực hiện chu kỳ thăm dò (*Polling*) ngắn 50 ms. Khoảng thời gian 50 ms là đủ nhỏ để mắt người cảm nhận sự đồng bộ tức thời giữa LED đơn và NeoPixel (độ trễ thị giác tối đa chỉ 0.05 giây), đồng thời vẫn giải phóng tới hơn 98% thời gian CPU nhờ `vTaskDelay(50)`.

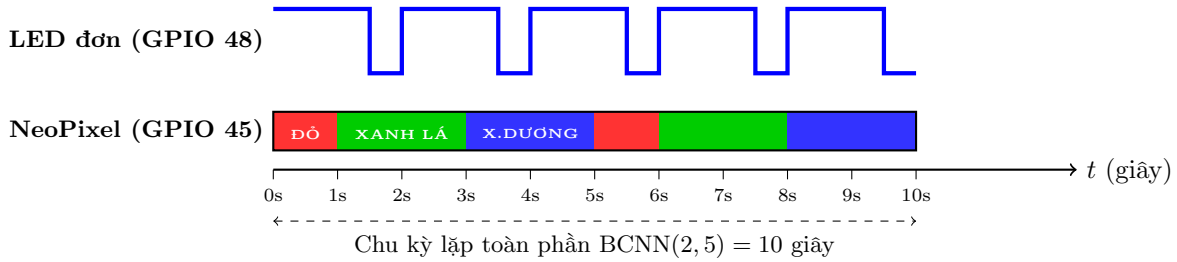
2.3 Bài 3: Các mẫu nhấp nháy độc lập (Independent Blink Patterns)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Hiện thực `task_led_blinky` với chu kỳ SÁNG 1,5 s / TẮT 0,5 s.**
- **Hiện thực `task_neo_blinky` để NeoPixel lần lượt chuyển qua ĐỎ (1 s), XANH LÁ (2 s) và XANH DƯƠNG (2 s), lặp lại liên tục.**
- **Cả hai tác vụ phải chạy hoàn toàn độc lập, bất kể trạng thái của tác vụ còn lại.**

2.3.1 Biểu đồ thời gian bất đồng bộ (Asynchronous Timing)

Chu kỳ của LED đơn là $T_{\text{LED}} = 1.5\text{s} + 0.5\text{s} = 2.0\text{s}$. Chu kỳ của NeoPixel là $T_{\text{NEO}} = 1\text{s} + 2\text{s} + 2\text{s} = 5.0\text{s}$. Bội chung nhỏ nhất của hai chu kỳ là $\text{BCNN}(2, 5) = 10\text{s}$. Sau mỗi 10 giây, toàn bộ mẫu kết hợp của cả hai tác vụ mới lặp lại vị trí pha ban đầu.



2.3.2 Mã nguồn hoàn chỉnh (`exercise_3.cpp`)

```cpp
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)
#define NEO_PIN 45          // NeoPixel data pin (GPIO 45)
#define LED_COUNT 1         // 1 LED NeoPixel

// Tac vu 1: Nhap nhay LED doc lap (SANG 1.5s, TAT 0.5s)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);
  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(1500); // Sang 1.5s

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(500);  // Tat 0.5s
  }
}

// Tac vu 2: NeoPixel chuyen mau doc lap (DO 1s -> XANH LA 2s -> XANH DUONG 2s)
void task_neo_blinky(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();
```

```
26
27    while (1) {
28      strip.setPixelColor(0, strip.Color(255, 0, 0)); // DO 1 giay
29      strip.show();
30      vTaskDelay(1000);
31
32      strip.setPixelColor(0, strip.Color(0, 255, 0)); // XANH LA 2 giay
33      strip.show();
34      vTaskDelay(2000);
35
36      strip.setPixelColor(0, strip.Color(0, 0, 255)); // XANH DUONG 2 giay
37      strip.show();
38      vTaskDelay(2000);
39    }
40  }
41
42  void setup() {
43    Serial.begin(115200);
44
45    xTaskCreate(
46        task_led_blinky,    // Ham tac vu
47        "Task_LED_Blinky", // Ten tac vu
48        2048,               // Stack depth (bytes)
49        NULL,               // Tham so truyen vao
50        1,                  // Do uu tien (Priority)
51        NULL                // Task handle
52    );
53
54    xTaskCreate(
55        task_neo_blinky,    // Ham tac vu
56        "Task_NEO_Blinky", // Ten tac vu
57        2048,               // Stack depth (bytes)
58        NULL,               // Tham so truyen vao
59        1,                  // Do uu tien (Priority)
60        NULL                // Task handle
61    );
62  }
63
64  void loop() {}
```

2.3.3 Phân tích kỹ thuật & Cơ chế FreeRTOS

- **Thực thi độc lập (Decoupled Execution):** Không có bất kỳ biến chung hay điểm nghẽn nào giữa hai tác vụ. Bộ lập lịch FreeRTOS duy trì hai đồng hồ định thời độc lập trong hàng đợi trễ (*Delayed Task List*).
- **Cơ chế Round-Robin trên cùng mức ưu tiên:** Cả hai tác vụ được gán cùng mức ưu tiên là 1 (`uxPriority = 1`). Khi cả hai cùng sẵn sàng (*Ready*), bộ lập lịch chia sẻ lát cắt thời gian (*Time-slicing*) công bằng thông qua ngắt SysTick, đảm bảo tính thời gian thực chính xác tuyệt đối mà không bên nào bị chiếm quyền quá mức.

2.4 Bài 4: Chuỗi màu NeoPixel nâng cao (Cầu vồng 5 màu, mỗi màu 3s)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Giữ `task_led_blinky` nhấp nháy theo mẫu SÁNG 2 s, TẮT 2 s.**
- **Tạo tác vụ mới `task_neo_rainbow` cho NeoPixel lần lượt chuyển qua năm màu khác nhau: ĐỎ, VÀNG, XANH LÁ, XANH NGỌC (Cyan), XANH DƯƠNG; mỗi màu duy trì đúng trong 3 giây.**
- **Tất cả tác vụ chạy đồng thời; LED và NeoPixel không được đồng bộ với nhau.**

2.4.1 Bảng ánh xạ mã màu RGB cho chuỗi 5 màu cầu vồng

STT	Tên màu hiển thị	Giá trị Red (R)	Giá trị Green (G)	Giá trị Blue (B)	Thời gian giữ
0	ĐỎ (Red)	255	0	0	3 giây
1	VÀNG (Yellow)	255	255	0	3 giây
2	XANH LÁ (Green)	0	255	0	3 giây
3	XANH NGỌC (Cyan)	0	255	255	3 giây
4	XANH DƯƠNG (Blue)	0	0	255	3 giây

Bảng 2: Bảng mã màu RGB và thời gian duy trì của chuỗi màu cầu vồng

2.4.2 Mã nguồn hoàn chỉnh (`exercise_4.cpp`)

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)
#define NEO_PIN 45          // NeoPixel data pin (GPIO 45)
#define LED_COUNT 1         // 1 LED NeoPixel

// Cau truc RGB Color
struct RGBColor {
  uint8_t r, g, b;
};

// Mang 5 mau cau vong
const RGBColor rainbowColors[5] = {
    {255, 0, 0},    // DO
    {255, 255, 0}, // VANG
    {0, 255, 0},    // XANH LA
    {0, 255, 255}, // XANH NGOC (Cyan)
    {0, 0, 255}     // XANH DUONG
};

// Tac vu 1: LED don nhap nhay SANG 2s, TAT 2s
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);
```

```
  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(2000);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(2000);
  }
}

// Tac vu 2: NeoPixel chuyen 5 mau cau vong, moi mau giu 3 giay (3000 ms)
void task_neo_rainbow(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  int colorIndex = 0;

  while (1) {
    RGBColor c = rainbowColors[colorIndex];
    strip.setPixelColor(0, strip.Color(c.r, c.g, c.b));
    strip.show();

    vTaskDelay(3000); // Giu 3 giay

    // Chuyen sang mau tiep theo tuan hoan
    colorIndex = (colorIndex + 1) % 5;
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(
      task_led_blinky,   // Ham tac vu
      "Task_LED_Blinky", // Ten tac vu
      2048,              // Stack depth (bytes)
      NULL,              // Tham so truyen vao
      1,                 // Do uu tien (Priority)
      NULL               // Task handle
  );

  xTaskCreate(
      task_neo_rainbow,  // Ham tac vu
      "Task_NEO_Rainbow", // Ten tac vu
      2048,              // Stack depth (bytes)
      NULL,              // Tham so truyen vao
      1,                 // Do uu tien (Priority)
      NULL               // Task handle
  );
}

void loop() {}
```

2.4.3 Phân tích kỹ thuật & Cơ chế FreeRTOS

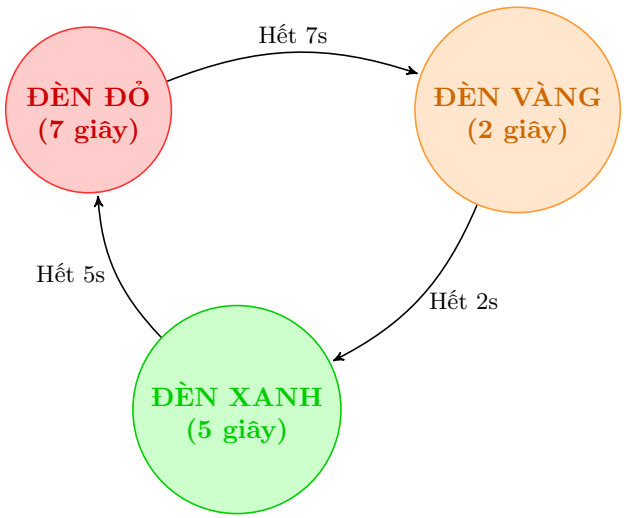
- **Mô hình lập trình hướng dữ liệu (Data-driven):** Thay vì viết mã lặp lại 5 lần lời gọi `strip.setPixelColor()` và `vTaskDelay()`, việc sử dụng cấu trúc `struct RGBColor` kết hợp mảng hằng số `rainbowColors[5]` và phép toán modulo `(colorIndex + 1) % 5` giúp mã nguồn cực kỳ tinh gọn, tối ưu kích thước bộ nhớ Flash và dễ mở rộng.
- **Tính toán tương quan chu kỳ:** Chu kỳ của chuỗi màu cầu vồng là $T_{\text{Rainbow}} = 5 \times 3\text{s} = 15\text{s}$. Chu kỳ của LED đơn là $T_{\text{LED}} = 2\text{s} + 2\text{s} = 4\text{s}$. Bội chung nhỏ nhất $\text{BCNN}(4, 15) = 60$ giây. Nghĩa là cứ sau đúng 1 phút, hai thiết bị ngoại vi mới quay lại cùng pha ban đầu (LED bắt đầu SÁNG đồng thời NeoPixel bắt đầu chuyển màu ĐỎ).

2.5 Bài 5: Mô phỏng hệ thống đèn giao thông với nhịp tim Heartbeat

Yêu cầu đề bài (Trích phiếu bài tập)

- **Tác vụ 1 (`task_led_blinky`): LED trên board đóng vai trò nhịp tim hệ thống (*Heartbeat*) — SÁNG 1 giây, rồi TẮT 1 giây, lặp liên tục.**
- **Tác vụ 2 (`task_traffic_light`): NeoPixel đóng vai trò đèn tín hiệu giao thông, lặp vô hạn theo chu kỳ:**
  - **ĐỎ: duy trì trong 7 giây.**
  - **VÀNG (AMBER): duy trì trong 2 giây.**
  - **XANH: duy trì trong 5 giây.**

2.5.1 Mô hình máy trạng thái hữu hạn (FSM) Đèn Giao Thông



2.5.2 Mã nguồn hoàn chỉnh (`exercise_5.cpp`)

```cpp
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)
#define NEO_PIN 45          // NeoPixel data pin (GPIO 45)
#define LED_COUNT 1         // 1 LED NeoPixel

// Cau truc pha den giao thong (Mau sac va thoi gian sang)
struct TrafficPhase {
  uint8_t r, g, b;
  uint32_t duration_ms;
};

// Chuoi trang thai den giao thong: DO 7s -> VANG 2s -> XANH 5s
const TrafficPhase trafficPhases[3] = {
    {255, 0, 0, 7000},   // Den DO: 7 giay
    {255, 180, 0, 2000}, // Den VANG (Amber): 2 giay
```

```
    {0, 255, 0, 5000}     // Den XANH: 5 giay
};

// Tac vu 1: Nhip tim he thong (SANG 1 giay / TAT 1 giay)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    digitalWrite(LED_PIN, HIGH); // Heartbeat SANG 1s
    vTaskDelay(1000);

    digitalWrite(LED_PIN, LOW);  // Heartbeat TAT 1s
    vTaskDelay(1000);
  }
}

// Tac vu 2: Den giao thong tuan tu theo chu ky
void task_traffic_light(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  int phase = 0;

  while (1) {
    TrafficPhase p = trafficPhases[phase];
    strip.setPixelColor(0, strip.Color(p.r, p.g, p.b));
    strip.show();

    vTaskDelay(p.duration_ms); // Giu mau theo thoi gian quy dinh cua tung pha

    // Chuyen sang pha tiep theo trong chu ky (Do -> Vang -> Xanh -> Do...)
    phase = (phase + 1) % 3;
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(
      task_led_blinky,       // Ham tac vu
      "Task_LED_Heartbeat", // Ten tac vu
      2048,                  // Stack depth (bytes)
      NULL,                  // Tham so truyen vao
      1,                     // Do uu tien (Priority)
      NULL                   // Task handle
  );

  xTaskCreate(
      task_traffic_light,    // Ham tac vu
      "Task_Traffic_Light", // Ten tac vu
      2048,                  // Stack depth (bytes)
      NULL,                  // Tham so truyen vao
      1,                     // Do uu tien (Priority)
      NULL                   // Task handle
  );
```

```
75 }
76
77 void loop() {}
```

2.5.3 Phân tích kỹ thuật & Ý nghĩa hệ thống

- **Cấu trúc `TrafficPhase`:** Việc thiết kế cấu trúc dữ liệu gồm cả 3 kênh màu RGB và tham số thời gian `duration_ms` biến tác vụ `task_traffic_light` thành một bộ thực thi bảng điều khiển (*Table-driven state machine*). Điều này giúp dễ dàng tái cấu hình thời gian đèn (ví dụ kéo dài đèn đỏ giờ cao điểm) mà không cần can thiệp logic mã lệnh.

- **Ý nghĩa của tín hiệu nhịp tim (*Heartbeat*):** Trong các thiết bị nhúng công nghiệp và IoT thực tế, khi hệ thống gặp lỗi treo (Deadlock, HardFault hoặc rò rỉ bộ nhớ), bộ vi điều khiển thường bị đơ. Tín hiệu LED nhấp nháy 1s/1s đều đặn là minh chứng trực quan xác nhận bộ lập lịch FreeRTOS đang hoạt động hoàn toàn khỏe mạnh. Nếu đèn nhịp tim ngừng nháy, kỹ sư lập tức nhận biết hệ thống đã gặp sự cố nghiêm trọng mà không cần mở máy tính gỡ lỗi.

3 Kết Luận và Bài Học Kinh Nghiệm (Lab 2)

Qua quá trình thực nghiệm 5 bài tập của Bài thực hành 2 trên bo mạch YoloUNO (ESP32-S3), sinh viên đã rút ra được các kết luận và bài học quan trọng:

1. **Ưu thế vượt trội của RTOS so với Super-loop:** Trong kiến trúc truyền thống không dùng RTOS, việc điều khiển đồng thời hai thiết bị có chu kỳ không đồng nhất (như LED 2s và NeoPixel 5s hoặc 14s) đòi hỏi người lập trình phải tự quản lý biến đếm thời gian với hàm `millis()` rất phức tạp và dễ lỗi. FreeRTOS đã giải quyết triệt để vấn đề này bằng mô hình đa tác vụ với cú pháp `vTaskDelay()` tự nhiên, tường minh.
2. **Kỷ luật bộ nhớ ngăn xếp (Stack Management):** Việc cấp phát Stack Depth là 2048 bytes cho mỗi tác vụ điều khiển LED là an toàn, tránh được lỗi tràn ngăn xếp (*Stack Overflow*) thường gặp trên vi điều khiển.
3. **Đồng bộ hóa đúng cách:** Khi chia sẻ dữ liệu giữa các tác vụ, từ khóa `volatile` là bắt buộc để ngăn chặn sự tối ưu hóa sai của trình biên dịch trong môi trường đa luồng.

Tài liệu

[1] CoreIOT Documentation. (2024). *CoreIOT / ThingsBoard Platform Architecture and APIs*.

[2] Espressif Systems. (2023). *ESP32-S3 Series SoC Technical Reference Manual*.

[3] Real Time Engineers Ltd. (2024). *FreeRTOS Kernel Developer Guide*.

[4] Khoa KH&KT Máy tính - ĐH Bách Khoa ĐHQG-HCM. (2026). *Tài liệu hướng dẫn thực hành IoT trên YoloUNO (ESP32-S3)*.